

Beyond Size: Do Hybrid PQC Certificates Actually Enforce the Classical–PQC Binding? A Cost-and-Security Study

Minwoo Lee¹ , Minjoo Sim¹ , Siwoo Eum¹ , Subeen Cho² , Yulim Hyoung² , and Hwajeong Seo² 

¹ Department of Information Computer Engineering, Hansung University, Seoul, Republic of Korea

{minunejip, minjoos9797, shuraatum}@gmail.com

² Department of Convergence Security, Hansung University, Seoul, Republic of Korea
{chosubin1208, yulim4hyoung, hwajeong84}@gmail.com (corresponding author: Hwajeong Seo)

Abstract. As TLS 1.3 migrates to post-quantum cryptography (PQC), hybrid X.509 transition strategies – alternative-signature (Catalyst), Composite, Chameleon, and signature combiners – are compared on cost but rarely on whether they actually *enforce* the classical↔PQC binding they promise. We show they often do not, and that the failure persists even in stacks that *do* check the binding. The *same* BouncyCastle library accepts a Catalyst certificate carrying a forged ML-DSA signature on its default path yet rejects it via an opt-in call; and wolfSSL, the only stack that checks a present alternative signature by default, cannot *require* one, so a stripped certificate is silently accepted (a path whose experimental build also shipped CVE-2026-5393). That no stack can *mandate* the binding is the surprise; that an ignored non-critical extension is skipped by nine verifiers is the expected X.509 baseline, which we reproduce with an independent non-BouncyCastle generator. Composite, by contrast, binds structurally (three verifiers across three OID families reject corruption of either half). Folding this *measured* enforcement axis into a four-axis cost model (strategy sizes differ by under 4.4%) overturns the cost-led recommendation: once enforcement is priced in, the compatibility winner Catalyst is displaced by a structurally-bound encoding – a combiner where the operator controls both endpoints, or Composite once a like-for-like P-256 pairing ships. A compatibility-led ranking can thus recommend a strategy whose PQC protection no deployed verifier enforces.

Keywords: Post-quantum cryptography · X.509 · TLS 1.3 · Hybrid certificates · ML-DSA · Crypto agility.

1 Introduction

In preparation for cryptographically relevant quantum computers, the adoption of post-quantum cryptography (PQC) has become a pressing task. In 2024 NIST

standardised the lattice-based signature scheme ML-DSA as FIPS 204 [1], and the migration of existing public-key infrastructure to PQC is now underway. Certificate migration, however, lags key exchange: a 2026 scan of 32,011 domains found 49% hybrid PQ key-exchange adoption but 0% hybrid PQ certificate adoption [2] – authentication is where the transition has barely begun, so the strategy choice is still open, being decided now, before deployment locks it in.

A central challenge is *crypto agility* – switching algorithms without operational disruption [3]. PQC migration touches certificate formats, CA infrastructure, and client compatibility; the lower the switching cost, the higher the agility.

In TLS 1.3, server authentication relies on X.509 certificates [4]: migrating their signature to PQC affects the public key, the CA signature, and the `CertificateVerify` signature at once. The difficulty is size – ML-DSA-65 (NIST Level 3) inflates a certificate from 377 to 5,507 bytes³ ($\approx 14.6\times$), negligible on broadband but costly on mobile, IoT, and in-vehicle (CAN-FD) links.

To address this, several transitional strategies exist. Hybrid-certificate formats that modify the X.509 structure include Catalyst [5], Composite [6], and Chameleon [7]; signature combiners that work at the signature level without changing the format include the Naive Combiner and the Compact Combiner (Bird of Prey) [8]. No prior work compares these two families under one quantitative criterion (Chen [9] omits combiners; Janneck [8] omits cost; §2.2).

Starting point. A four-axis transition-cost model – certificate size, handshake bytes, verification time, and compatibility – was introduced in our prior work [10] to compare these strategies and derive per-environment recommendations. That model, like the comparative studies around it, scores security only through a qualitative *compatibility* proxy and never checks whether a strategy’s post-quantum protection is actually enforced.

Contributions. This paper supplies that missing security dimension:

1. We *measure* binding enforcement under real verifiers and find it fails *even where a verifier checks the PQC half*. The *same* BouncyCastle library accepts a forged Catalyst certificate on its default path yet rejects it through its opt-in API – same library, same certificate, opposite verdicts – and wolfSSL, the only stack that checks a present alt-signature by default, cannot *require* one, so a stripped certificate is accepted (it also shipped CVE-2026-5393 in that path). That the non-critical extension is *ignored by default* across nine standard verifiers is the expected X.509 baseline (confirmed with an independent, non-BouncyCastle generator); the enforcement gap above it is predicted by no specification. Composite, by contrast, binds structurally: three verifiers across three OID families reject corruption of *either* half.
2. We add binding enforcement as a fifth axis. As a hard *constraint* – discard strippable strategies, then rank the rest on cost – it removes Catalyst from the top rank independent of any weight; an additive form reproduces this at a

³ CA-issued strategy certificates; the self-signed single-algorithm certificates of Table 2 differ by a few bytes (387 / 5,510).

moderate security weight. The replacement is a structurally-bound encoding (a combiner in controlled-endpoint deployments; Composite once a like-for-like P-256 pairing ships) – a trade-off the cost-led ranking hid.

Pricing enforcement in *overturns the prior model’s cost-led recommendation* [10]: treating an alternative-signature certificate as a security-equivalent hybrid is a category error only a measured axis exposes.

2 Background and Related Work

In TLS 1.3 [4] a client verifies a certificate’s CA signature and the server’s live `CertificateVerify` – three verifications for a length-2 chain; PQC migration replaces all three, inflating both certificate size and verification time.

2.1 PQC Certificate Transition Strategies

Strategies are hybrid-certificate formats or signature combiners. *Catalyst* [5] (ITU-T X.509) carries the PQ key and signature as *non-critical* extensions, so legacy clients ignore them and validate the classical signature alone (maximal compatibility). *Composite* [6] fuses both under one OID, requiring *both* (“AND”) but unparseable by legacy clients. *Chameleon* [7] embeds a full PQC “Delta” certificate in a non-critical extension (detachable, verified Base-only like Catalyst; the largest format). *Combiners* keep the format: the Naive one concatenates both signatures (no SUF), and Janneck’s [8] Compact Combiner restores strong unforgeability via an EdDSA Fiat–Shamir structure.

2.2 Related Work

Prior comparisons are partial: Chen [9] ranks the three certificate formats by size/compute but omits combiners; Janneck [8] proves a combiner’s unforgeability without cost analysis; Kampanakis et al. [11] and Fan et al. [12] study PQ-certificate viability and PKI impact, not cross-strategy. Closest to our angle, Ricchizzi et al. [13] *generate* and validate Catalyst/Composite/Chameleon certificates (ML-DSA/SLH-DSA, BouncyCastle) and confirm interoperability – functional *support*; we instead measure whether deployed verifiers *enforce* the binding (rejecting a forged or stripped PQC half), which a functional study does not test. A separate line *defines* the *non-separability* of hybrid signatures against stripping [14,15,16], and the draft we build on [17] flags stripping but delegates it to applications; we instead *measure* where real verifiers fall – the alternative-signature family is separable in practice (its PQC half ignored) while Composite is not. A concurrent TLS 1.3 draft [18] makes the “verify both, no silent fallback” principle normative for the related dual-*certificate* mechanism (both signatures MUST validate; silent single-certificate fallback is a stated downgrade risk) – protocol-layer grounding for what we measure, at the certificate-path layer, in deployed verifiers. Building on the four-axis cost model [10], we unify the two strategy families and add a measured security axis.

3 Transition-Cost Model

To quantify the crypto agility of a transition strategy, our prior work [10] defines a four-axis transition-cost model, which we adopt and recall here; a lower cost means higher crypto agility.

The four axes are C_{size} (DER byte size of one certificate), C_{hs} (TLS 1.3 authentication bytes; $2C_{\text{size}} + \sigma_{\text{CertVerify}}$ for a length-2 chain), C_{verify} (client chain-verification time, μs), and C_{compat} (a 0–1 legacy-compatibility score combining standardisation, library support, and fallback ability, set per environment).

For strategy S in environment E , the total transition cost is the weighted sum

$$C_{\text{total}}(S, E) = w_1 \hat{C}_{\text{size}} + w_2 \hat{C}_{\text{hs}} + w_3 \hat{C}_{\text{verify}} + w_4 \hat{C}_{\text{compat}}, \quad (1)$$

where each $\hat{C}$ is a per-axis normalised cost (size relative to the classical baseline, verification min–max-normalised, handshake a weighted combination) and the weights (w_1, w_2, w_3, w_4) over (size, handshake, verify, compatibility) depend on the environment – server farm (0.10, 0.20, 0.20, 0.50), mobile (0.20, 0.30, 0.20, 0.30), IoT (0.35, 0.20, 0.35, 0.10) – adopted from [10] unchanged, including its property that the axes are not all on an identical $[0, 1]$ scale. Server farms have ample bandwidth, so compatibility dominates; mobile raises w_2 (user-visible handshake latency); IoT controls both endpoints, so compatibility matters least.

The compatibility cost C_{compat} (0–1) is environment-dependent: combiners lack standardised libraries, scoring 0.8/0.9 in general (server-farm/mobile) settings but 0.3 in IoT, where the operator controls both endpoints; Catalyst is the reverse (0.1/0.1 general, 0.3 IoT, its legacy fallback being needless there); Chameleon tracks Catalyst (0.2/0.2/0.3) and Composite sits between (0.7/0.8/0.4). These hand-set scores, from [10], are stress-tested in §4.6, where Section 4 adds a fifth, measured axis C_{bind} (binding enforcement), weighted w_5 with w_1 – w_4 rescaled to $(1 - w_5)$.

4 Is the Classical–PQC Binding Actually Enforced?

The cost model of Section 3 treats security only qualitatively, through C_{compat} . Yet its own conclusion is that strategy choice should be driven by “compatibility and the security model”. This section measures the security side of that claim: for each hybrid strategy, does a real verifier *enforce* the classical $\leftrightarrow$ PQC binding, or is the PQC half ignorable? We find that enforcement is a *structural* property of the encoding, not of the algorithm, and that adding it as a fifth cost axis overturns the prior ranking.

4.1 Threat model

PQC migration presumes the classical signature will eventually be forgeable. The relevant adversary can therefore forge ECDSA/Ed25519 (including a CA’s

classical signature) but cannot break ML-DSA. The question is whether the PQC half still protects the certificate. In our experiments we hold the test-CA and leaf keys; re-signing with them models exactly this adversary’s classical-forgery capability.

We separate two layers. At the *verifier* layer – our focus – the question is whether a verifier checks a hybrid certificate’s PQC component; Composite forces this, Catalyst does not. A full *downgrade* to a plain classical certificate is a distinct policy-layer question no encoding prevents without a require-PQC policy (addressed at the connection layer by, e.g., cached PQC-availability commitments [19]). The layers interact: a require-PQC policy of the form “the leaf must present a PQC algorithm” is satisfied by a Catalyst certificate at the *parse* level – the extension is present but never validated – so the policy passes while the binding goes unenforced, and Composite closes that gap for free. Such policies are only now emerging (certificate adoption is still $\approx 0\%$ [2]), so it is a gap to foreclose before deployment. Our contribution is the verifier layer: we test whether the PQC half can be stripped or forged while the classical half stays valid – the property that matters once the classical signature is forgeable – so we tamper with the PQC, not the classical, half.

4.2 Method

We generate Catalyst (alternative-signature extensions) and Composite (single composite OID) certificates with BouncyCastle 1.81 [20] and oqsprovider 0.11.0 [21] (on OpenSSL 3.6), using ML-DSA-44 for this deep enforcement matrix (ML-DSA-65 on the second toolchain, §4.4). Binding enforcement is *algorithm-independent*: identical across ML-DSA, SLH-DSA and Falcon (supplementary), so it carries over to the ML-DSA-65 parameters of the cost axes (§3). To rule out a generator-specific artifact we also use an *independent, non-BouncyCastle* generator (Python `asn1crypto` assembling the `PreTBSCertificate`; OpenSSL-native ML-DSA produces a *real* alternative signature). We then verify (i) an untouched certificate, (ii) one whose PQC half is forged while the classical half stays valid, and (iii) one whose classical half is broken (control). Two points matter. First, the primary signature covers the whole `tbsCertificate` (including `altSignatureValue`), so we give each test certificate a valid classical primary over the forged PQC material: a rejection can then only be genuine PQC-binding enforcement, not collateral primary breakage. Second, the cert under test is always a non-anchor leaf whose signature is actually checked. The non-BouncyCastle Catalyst certificate is itself accepted by both alt-aware verifiers (wolfSSL and BouncyCastle’s opt-in API), so the silent-acceptance results below are not a single generator’s artifact. Each of the nine standard verifiers is *run* on these specimens with captured output (exact versions in the supplementary); the captured verifiers (OpenSSL 3.0.13 among them) and an earlier OpenSSL 3.6.2/BouncyCastle 1.79 configuration behave identically, so the result is not version-specific. Composite is tested by corrupting the ML-DSA component, and separately the classical component, of its single signature.

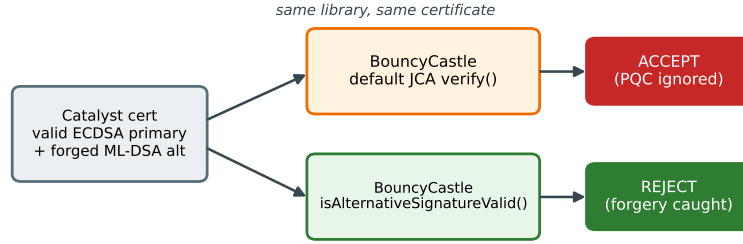


Fig. 1. Capability is not enforcement. A single forged-PQC Catalyst certificate (valid ECDSA primary, forged ML-DSA alternative) yields opposite verdicts from the *same* BouncyCastle library: the default JCA `verify()` path accepts it (the alternative signature is never checked), while the opt-in `isAlternativeSignatureValid()` call catches the forgery. The accepting path is the one TLS stacks use by default.

4.3 Result: enforcement is structural

With Catalyst, a certificate carrying a *valid* ECDSA signature and an invalid PQC alternative signature is accepted both by OpenSSL’s default `verify` (garbage alternative-signature extension) and by BouncyCastle’s default JCA `verify()` path (a real but forged ML-DSA alternative signature): both validate only the classical primary and ignore the PQC half. This is the *expected* X.509 baseline – a non-critical extension is ignored by mandate – and it holds across *nine* standard verifiers (OpenSSL, Go, Botan, GnuTLS, mbedTLS, NSS, Windows CryptoAPI, Java’s SUN and BouncyCastle paths); most parse the extension and ignore it, while mbedTLS reaches the same accept by not supporting alt-signatures at all. A controlled flip pins it to structure: the *same* content marked *critical* is rejected at parse (Go/Botan: “unhandled critical extension”), so the criticality bit alone flips accept to reject – enforcement is a property of the certificate, not the library or algorithm. The *surprising* results are where a verifier *does* engage the alternative signature yet still fails to enforce. The *same* BouncyCastle that ignores it by default rejects the same forgery through its opt-in API: capability without enforcement (Fig. 1) – same library, same certificate, opposite verdicts, and the default path TLS stacks use is the one that does not enforce (wolfSSL is the other case, §4.4). With Composite, corrupting *either* the ML-DSA or the classical component of the composite signature is rejected with a signature-verification failure – both halves are inside the one signature that is checked, so neither can be ignored; this holds across three independent verifiers spanning three disjoint composite-OID families (oqsprovider, BouncyCastle, and the draft-lamps reference implementation). Enforcement is thus strong, but those three families do not cross-verify (each rejects the others’ OID), so Composite’s binding is robust while its standardisation is still immature.

Table 1. Ecosystem awareness and *require*-mode survey for the alternative-signature binding (second toolchain, as of 2026-06; binding verdicts agree with the primary deep matrix). The deep enforcement breadth (nine standard stacks) is reported in §4.3 and the supplementary.

Verifier	Surfaces alt-sig	Verifies by default	“require” mode
OpenSSL 3.6.2	yes (by name)	no	none
BouncyCastle 1.79	yes	no	opt-in API only
wolfSSL 5.9.1 (exp.)	yes	yes, but strippable	none
GnuTLS 3.8.12	no (hidden)	no	none
NSS	numeric OID only	no	none
Python <code>cryptography</code> 49	raw blob	no	none
mbedtls	no (unsupported)	no	none

4.4 The enforcement landscape across the ecosystem

The silent acceptance above already spans nine enforcement-tested stacks; here we ask a complementary question – how does the ecosystem *surface* the extensions, and does any stack offer a way to *require* them? We surveyed seven stacks for this (Table 1; the binding verdicts match the measured runs of §4.3). The counts index one 162-cell matrix over 15 stacks (measured enforcement runs plus algorithm/criticality variants that follow from an alt-blind verifier never reading the extension): *nine* are alt-blind (§4.3), *seven* are surveyed for require-mode (Table 1), and *three* independent verifiers confirm Composite – overlapping subsets, not a partition. The result is systemic and, unlike the two-library result, *not* predictable from the specification. **(i)** None of the 15 stacks we tested provides a working default – or even readily configurable – way to *require* the binding (BoringSSL and rustls remain future work); enforcement is at best an explicit per-call opt-in (BouncyCastle). **(ii)** Awareness is inconsistent, from naming the extensions (OpenSSL) to no support (mbedtls; Table 1). **(iii)** The one stack with dual-algorithm certificate support, wolfSSL, *does* verify a present alt-signature – a leaf with a forged ML-DSA alternative signature is rejected (measured against a CA chain) – yet there is no mechanism to mark an alt-signature as *required*, so a *stripped* classical-only leaf, indistinguishable to the verifier from an ordinary single-algorithm certificate, is accepted. wolfSSL thus checks a present binding but cannot mandate one. The same experimental dual-algorithm path also shipped **CVE-2026-5393** [22]: an out-of-bounds read in `CertificateVerify` processing, scored 9.1 Critical by NVD but 6.3 Medium by wolfSSL (a known NVD/CNA scoring gap), patched only in the 5.9.1 build we evaluate. This memory-safety defect is independent of the enforcement gap, but together they show that even the one stack attempting alt-signature verification ships that capability as fragile, experimental code.

4.5 The default-ignore baseline is by design (the gap above it is not)

The standard intends the *baseline* of §4.3 exactly: the alternative-signature extensions are non-critical, so a non-upgraded system treats the certificate as conventional [17], and the draft only *recommends* (SHOULD, not MUST) rejecting a certificate whose issuer has an alternative signature, delegating “stripping attacks” to “dependent protocols and applications” [17]. Catalyst – and the delta-certificate Chameleon (its non-critical delta likewise ignored, measured on Go and Botan) – thus offer a migration aid, not a security guarantee; a compatibility-led ranking that treats them as security-equivalent hybrids gets this wrong.

4.6 A fifth axis, and a re-ranking

We add a binding-enforcement axis $C_{\text{bind}} \in [0, 1]$ (lower is better): 0 when the binding is structurally enforced (Composite, measured; the Naive Combiner, demonstrated by construction; the Compact Combiner, by its strong-unforgeability proof [8]), 1 when no verifier *requires* the PQC half so it can be stripped (Catalyst, measured: ignored by default, and strippable even by wolfSSL which does verify a present alt-signature; Chameleon, delta-blind, measured on Go and Botan and structural across the other alt-blind stacks; the binary captures “can the binding be stripped”, collapsing the verifier spectrum of §4.4). Because C_{bind} is binary, an additive re-ranking is almost mechanical – weight the losing axis enough and the winner flips – so we take the *constraint* reading as primary: discard the strippable strategies, then rank the rest on cost. Only the structurally-bound strategies survive once a transitional adversary is in scope, and among them Composite is cost-competitive at a like-for-like P-256 level (§5.3); this needs no weight. For continuity we *also* price the axis in additively (weight w_5 , w_1 – w_4 rescaled to $(1 - w_5)$; Fig. 2); the like-for-like costs also *correct* the prior model, where Composite ranked *last* only because of its P-384 half. At $w_5=0$ the ranking is compatibility-led (Catalyst first; Composite now mid-pack, 3rd, not last); as w_5 grows Catalyst loses the top rank to *Composite* once $w_5 \geq 0.23$ (server farm) and ≥ 0.17 (mobile), IoT staying a near-tie – robust to *joint* ± 0.1 perturbation of all compatibility scores and weights (512 corners; Catalyst never retains the top rank), surviving only under an implausible ± 0.3 .

One caveat keeps this honest: the winning Composite is the like-for-like P-256 pairing, which *no provider currently ships* (§5.3), so this is guidance for *when* it ships, not a claim that an off-the-shelf Composite wins. Since size and verification barely separate the strategies (both within $\sim 4\%$), the decision reduces to *compatibility versus binding enforcement*; we synthesise each option’s deployment-readiness in §6.

Our enforcement results above are offline tamper tests; as supporting evidence that the gap is reachable end-to-end, a live TLS 1.3 handshake has an OpenSSL `s_client` accept a server presenting a PQC-stripped, classical-only certificate with `Verify return code: 0 (ok)`, identical to the honest hybrid (server flight 14,599 vs 18,411 B). This particular demo is the policy-layer

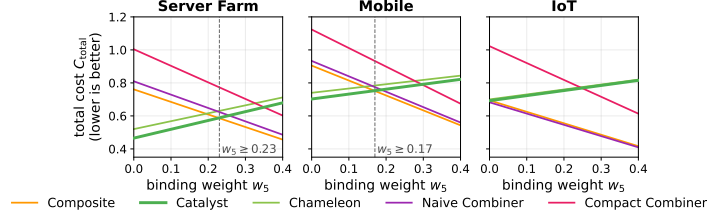


Fig. 2. Continuity check for the weight-independent constraint result (Catalyst leaves the top rank once strippable strategies are discarded, no weight needed). Pricing C_{bind} in additively corroborates it: total cost C_{total} versus weight w_5 (like-for-like costs; lower is better; Catalyst in bold), with *Composite* overtaking Catalyst at $w_5 \geq 0.23$ (server farm) / 0.17 (mobile, dashed line); IoT shows no crossover, a strip-proof combiner already leading.

strip-to-classical that no encoding – Composite included – prevents without a require-PQC policy; it shows the path is reachable on a live connection, not that any encoding closes it here. The full tamper matrix (162 cells over 15 stacks, four mechanisms, multi-level chains, network-emulated handshake) is provided as supplementary material.

Limitations. The C_{bind} values are established by measurement (Catalyst, Composite, Chameleon) or, for the combiners, by construction/proof (Naive; Compact [8]). The weight-independent “Catalyst loses the top rank” result rests on the *measured* $C_{bind}=1$; which structurally-bound strategy then leads among the survivors (a combiner in IoT, Composite in server/mobile) is decided by the estimated cost axes. Crucially, the combiners’ $C_{bind}=0$ is a *structural prediction* – no library yet parses a combiner certificate, so it carries the same capability-vs-enforcement caveat we raise for Catalyst, pending a verifier-level measurement. C_{bind} is binary here; a finer metric separating verifier-level enforcement from a policy-level “require-PQC” mode is future work.

5 Evaluation

5.1 Experimental Setup

Experiments were run in the Open Quantum Safe (OQS) Docker image (openquantumsafe/oqs-oss13), with liboqs 0.15 and oqsprovider 0.11.0 on OpenSSL 3.6 for PQC certificate generation and signing-performance measurement; signing performance was measured with `openssl speed` over 3-second runs [23]. All five strategies use ML-DSA-65 (NIST Level 3), paired with P-256 (Catalyst, Chameleon, Naive Combiner), Ed25519 (Compact Combiner), or P-384 (Composite, matching Level 3 on the classical side via oqsprovider’s `p384_mldsa65`); the P-384 choice raises Composite’s verification time (Sec. 5.3). Catalyst and Composite certificates are measured; Chameleon and the two

Table 2. Single-algorithm certificate cost across the NIST signature families (self-signed cert DER size; real `X509_verify`, OpenSSL 3.6+oqsprovider). *Within* ML-DSA-65 the five hybrid strategies differ only 4.4% (above); *across* families cost varies $\approx 114\times$ – the algorithm dominates, the strategy is orthogonal.

Algorithm	Family (NIST level)	Cert (B)	Verify (μ s)
Ed25519	classical	316	53
ECDSA-P256	classical	387	37
ECDSA-P384	classical	448	353
Falcon-512	FN-DSA (L1)	1,795	335
Falcon-1024	FN-DSA (L5)	3,308	350
ML-DSA-44	FIPS 204 (L1)	3,981	316
ML-DSA-65	FIPS 204 (L3)	5,510	356
ML-DSA-87	FIPS 204 (L5)	7,468	357
SLH-DSA-128s	FIPS 205 (L1)	8,129	505
SLH-DSA-128f	FIPS 205 (L1)	17,361	777
SLH-DSA-192f	FIPS 205 (L3)	35,954	952

combiners are ASN.1-structure estimates, cross-checked against the measured Composite and Catalyst certificates (the dominant ML-DSA-65 key and signature are fixed-size).

5.2 Certificate Size

With ML-DSA-65 a single certificate grows from 377 B (ECDSA) to 5,507 B ($14.6\times$); the five hybrid strategies then cluster tightly in 5,579–5,823 B (Compact Combiner, Naive, Composite, Catalyst, Chameleon, in increasing order; measured for Catalyst/Composite, ASN.1 estimates otherwise), a maximum spread of only 4.4% because ML-DSA-65’s 1,952-B key and 3,309-B signature dominate. So strategy choice is driven by compatibility, security model, and operability rather than size. This clustering is ML-DSA-specific, though: across the NIST signature families and levels (Table 2; self-signed, so bytes differ slightly from the CA-issued certs above) certificate cost spans $\approx 114\times$ in size and $\approx 26\times$ in verification: the *algorithm*, not the hybrid strategy, drives cost.

5.3 Verification Time

The prior model sums `openssl speed` primitive verifies (a length-2 chain is three verifications; Catalyst’s legacy mode is plain ECDSA). Real verification (`X509_verify`, $N=3000$; run-to-run std $\leq 4.2\%$ over 20 runs), measured here for the first time, tracks it for classical keys (ECDSA-P256 37.2 vs 35.1 μ s) but runs $\approx 2\times$ higher for PQC certificates (P-384 composite 755 μ s) – per-call processing the benchmark omits. This is a near-constant additive per-call overhead – roughly $2\times$ the modeled cost for these tightly-clustered PQC strategies, which share the ML-DSA-65 core – so it shifts every PQC strategy alike and cannot

Table 3. Per-environment cost ranking (C_{total} , $w_5=0$; like-for-like costs, Composite at P-256, whose verify is decomposed; lower is better). The PQC verify axis is uniformly $\sim 2\times$ under-modeled, so it shifts no PQC strategy’s rank.

Server farm			Mobile			IoT		
#	Strategy	C	#	Strategy	C	#	Strategy	C
1	Catalyst	0.465	1	Catalyst	0.702	1	Naive C.	0.682
2	Chameleon	0.520	2	Chameleon	0.740	2	Catalyst	0.692
3	Composite	0.761	3	Composite	0.905	3	Composite	0.694
4	Naive C.	0.810	4	Naive C.	0.933	4	Chameleon	0.700
5	Compact C.	1.004	5	Compact C.	1.123	5	Compact C.	1.023

reorder them. Composite’s higher *modeled* verify over the other PQ paths (the measured gap is the $1.9\times$ below) is not intrinsic but a curve choice: oqsprovider pairs ML-DSA-65 only with P-384, whose verification is far slower than P-256 (OpenSSL’s P-256-only assembly path; Table 2). Decomposed from those single-algorithm costs, a P-256/ML-DSA-65 composite (size 5,679 B *measured*; no p256_mldsa65 ships) verifies at $\approx 393 \mu\text{s}$ (ECDSA-P256 37 + ML-DSA-65 356, the 20-run mean the ranking uses) versus the measured 755 μs at P-384 – $1.9\times$ cheaper and matching Catalyst’s PQ path – so at a like-for-like L1-classical level Composite is cost-competitive (§4.6).

5.4 Per-Environment Suitability

With like-for-like costs (Table 3), Catalyst wins in server-farm and mobile (high compatibility weight rewards its legacy fallback), Chameleon second, and *Composite now third – no longer last* (its prior bottom rank was the P-384 verify artifact); in IoT the Naive Combiner edges a four-way near-tie. Once binding enforcement is priced in (§4.6), Composite overtakes Catalyst at a moderate security weight. (The Compact Combiner ranks last only because min–max normalisation magnifies Ed25519’s $\sim 16 \mu\text{s}$ verify *deficit* versus ECDSA-P256 – a real but tiny gap the scaling exaggerates.)

6 Conclusion

Building on a four-axis transition-cost model [10] (size, handshake, verify, compatibility) over three hybrid-certificate formats and two combiners, we asked whether the security side that model proxies qualitatively actually holds.

Compatibility, however, is not security. We measured whether the verifiers enforce the classical $\leftrightarrow$ PQC binding and found they do so only when the encoding is structural: Composite rejects a tampered ML-DSA half (three independent verifiers), whereas nine standard verifiers silently accept a Catalyst certificate with a garbage or forged alternative signature – by design of a standard that only *recommends* the check. Treating enforcement as a hard constraint (equivalently,

a fifth cost axis) removes Catalyst from the top rank. The honest diagnosis: today no strategy satisfies compatibility, enforced binding, and library support at once – the cost-winning P-256 Composite does not ship, the deployed P-384 Composite is a cost laggard, and no library yet parses a combiner certificate. Our guidance is forward-looking: where a transitional adversary is in scope, choose a structurally-bound encoding – Composite first (the only survivor with *measured* enforcement, confirmed across three independent OID families, though those families do not yet interoperate), once it matures; a combiner (construction-bound, awaiting a verifier) where the operator controls both endpoints – paired with a require-PQC policy, since no encoding alone defeats a full classical downgrade. Do not treat Catalyst’s compatibility advantage as a security equivalence.

Limitations and future work. The Chameleon and combiner *sizes* remain estimates; the binding scores are now all measured (Catalyst, Composite and Chameleon) or established by construction/proof (the combiners). The modeled verify cost is validated against real `X509_verify` (§5.3): accurate for classical, $\approx 2\times$ low for PQC certificates (uniform, ranking-preserving), and the Composite/P-384 mismatch is resolved by a P-256 composite (size measured, verification decomposed). The per-strategy ranking uses ML-DSA-65; a landscape across the NIST families (Table 2; ML-DSA/Falcon/SLH-DSA, levels 1–5) shows certificate cost varies $\approx 114\times$ by algorithm, so the ranking is algorithm-specific, whereas the binding dichotomy is algorithm-independent (confirmed across ML-DSA, SLH-DSA and Falcon). Future work includes extending the tamper-tested matrix to further high-traffic stacks (BoringSSL, rustls), end-to-end handshake *timing* and a direct PQC-certificate verify measurement to tighten the modeled $\approx 2\times$ underestimate, and per-algorithm *strategy* re-rankings beyond ML-DSA-65 (incl. KpqC schemes).

References

1. NIST: FIPS 204: Module-lattice-based digital signature standard (ML-DSA) (2024)
2. Dubey, V.M., Varshney, G.: Measurement study of post-quantum readiness of the internet: 2026. arXiv:2606.16473 (2026)
3. Bindel, N., Herath, U., McKague, M., Stebila, D.: Transitioning to a quantum-resistant public key infrastructure. In: Post-Quantum Cryptography (PQCrypto). pp. 384–405 (2017)
4. Rescorla, E.: The transport layer security (TLS) protocol version 1.3. RFC 8446 (2018)
5. ITU-T: X.509: Public-key and attribute certificate frameworks (2019)
6. Ounsworth, M., Gray, J., Pala, M., Klaufner, J.: Composite ML-DSA for use in internet PKI. IETF Internet-Draft, draft-ietf-lamps-pq-composite-sigs (2025)
7. Bonnell, C., Gray, J., Hook, D., Okubo, T., Ounsworth, M.: A mechanism for encoding differences in paired certificates. IETF Internet-Draft, draft-bonnell-lamps-chameleon-certs-07 (2025)
8. Janneck, J.: Bird of prey: Practical signature combiners preserving strong unforgeability. Cryptology ePrint Archive, Report 2025/1844 (2025)
9. Chen, A.C.H.: A comparative study of hybrid post-quantum cryptographic X.509 certificate schemes. arXiv:2511.00111 (2025)

10. Lee, M.W., Sim, M.J., Cho, S.B., Hyoungh, Y.L., Seo, H.J.: Comparative analysis of hybrid signature strategies for X.509 post-quantum transition. In: Proc. Conf. on Information Security and Cryptography–Summer (CISC-S), KIISC (2026)
11. Kampanakis, P., Panburana, P., Daw, E., Van Geest, D.: The viability of post-quantum X.509 certificates. Cryptology ePrint Archive, Report 2018/063 (2018)
12. Fan, J., Mister, S., Ounsworth, M.: Impact of post-quantum hybrid certificates on PKI. International Journal of Security and Networks (2021)
13. Ricchizzi, N., Schwinne, C., Pelzl, J.: Applied post-quantum cryptography: A practical approach for generating certificates in industrial environments. arXiv:2505.04333 (2025)
14. Hale, B., Ounsworth, M., et al.: Hybrid signature spectrums. IETF Internet-Draft, draft-ietf-pquip-hybrid-signature-spectrums (2024)
15. Bindel, N., Hale, B.: A note on hybrid signature schemes. Cryptology ePrint Archive, Report 2023/423 (2023)
16. Driscoll, F., Parsons, M., Hale, B.: Terminology for post-quantum traditional hybrid schemes. RFC 9794 (2025)
17. Truskovsky, A., Van Geest, D., Fluhrer, S., Kampanakis, P., Ounsworth, M., Mister, S.: Multiple public-key algorithm X.509 certificates. IETF Internet-Draft, draft-truskovsky-lamps-pq-hybrid-x509 (2018)
18. Shekh-Yusef, R., Tschofenig, H., Ounsworth, M., Reddy, K. T., Rosomakho, Y.: Post-quantum traditional (PQ/T) hybrid authentication with dual certificates in TLS 1.3. IETF Internet-Draft, draft-yusef-tls-pqt-dual-certs-01 (2025)
19. Sheffer, Y., Reddy, K. T.: PQC continuity: Downgrade protection for TLS servers migrating to PQC. IETF Internet-Draft, draft-sheffer-tls-pqc-continuity-02 (2026)
20. Legion of the Bouncy Castle: Bouncy castle cryptography library for Java. <https://www.bouncycastle.org>, v1.81 (deep matrix); v1.79 (second toolchain)
21. Open Quantum Safe Project: oqs-provider: OpenSSL 3 provider for post-quantum algorithms. <https://github.com/open-quantum-safe/oqs-provider>, v0.11.0, liboqs 0.15.0
22. MITRE: CVE-2026-5393: out-of-bounds read in wolfSSL dual-algorithm CertificateVerify processing. <https://nvd.nist.gov/vuln/detail/CVE-2026-5393> (2026), NVD CVSS 3.1 base 9.1 (Critical); fixed in wolfSSL 5.9.1
23. Open Quantum Safe Project: Open quantum safe. <https://openquantumsafe.org>